PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Регрессия в Scikit-learn

Содержание

- Идея урока
- Кейс урока
- Ключевые понятия
- Классификация и регрессия
- Основная цепочка работы
- Фрагменты кода
- Как понимать ошибку
- Почему прогноз не обязан совпадать с фактом
- Итог

Идея урока

Не во всех задачах достаточно выбрать один вариант из нескольких классов. Иногда требуется оценить величину, которая может принимать множество числовых значений. Например, важно узнать не просто, будет ли результат высоким или низким, а спрогнозировать конкретное количество очков.

В классификации модель выбирает класс, например friend / enemy или setosa / virginica. В регрессии модель предсказывает число: очки игрока, время выполнения, цену, температуру или другой количественный результат. На этом уроке обучающиеся запускают простую регрессионную модель в Scikit-learn, сравнивают предсказанные и фактические значения, а затем считают ошибку прогноза.

Предсказание модели необходимо проверять на данных, которые не использовались при обучении. Иначе нельзя понять, насколько хорошо модель справляется с новыми примерами, а не только запоминает известные ей наблюдения.

Кейс урока

ИИ должен предсказать новый результат игрока по двум признакам: времени тренировки и прошлому результату. Столбец score является числовой целевой переменной. Это не класс, а число, которое модель должна спрогнозировать.
Каждая строка таблицы описывает один пример: значения training_minutes и previous_score используются как входные данные, а score содержит правильный ответ для этого примера. Во время обучения модель ищет зависимость между признаками и целевой переменной.
Например, для игрока с определённым временем тренировки и прошлым результатом модель может спрогнозировать 71.3 очка. Регрессионный прогноз не обязан быть целым числом, даже если фактические результаты игроков записаны целыми числами.

Ключевые понятия

- регрессия — задача машинного обучения, где модель предсказывает числовое значение;
- признаки — данные, по которым модель строит прогноз; в этом кейсе признаками являются training_minutes и previous_score;
- целевая переменная — число, которое нужно предсказать; здесь это столбец score;
- LinearRegression — простая регрессионная модель из Scikit-learn, которая описывает зависимость целевого значения от признаков с помощью линейной функции;
- обучающая выборка — часть данных, на которой модель находит зависимость между признаками и результатом;
- тестовая выборка — отложенная часть данных, используемая для проверки модели после обучения;
- предсказание — числовой результат, который обученная модель вычисляет для набора признаков;
- ошибка прогноза — разница между фактическим и предсказанным значением;
- абсолютная ошибка — модуль разницы между фактическим значением и прогнозом, поэтому она не бывает отрицательной;
- MAE — средняя абсолютная ошибка; она показывает, на сколько единиц модель обычно ошибается.

Классификация и регрессия

Тип задачи определяется тем, какой ответ требуется получить. Если ожидается название категории, решается задача классификации. Если ожидается количественное значение, решается задача регрессии.

Тип задачи	Ответ модели	Пример
Классификация	класс	enemy, friend, setosa
Регрессия	число	72 очка, 18 минут, 4500 рублей

Важно различать числовую целевую переменную и номер класса. Если классы обозначены числами 0 и 1, задача всё равно может оставаться классификацией: эти числа служат названиями категорий, а не измеряемым результатом. В регрессии числовое значение имеет количественный смысл, поэтому разница между 70 и 75 очками равна 5 очкам.

Основная цепочка работы

Работа с регрессионной моделью состоит из последовательных этапов. Разделение данных особенно важно: модель обучается на одной части таблицы, а проверяется на другой.

1. загрузить таблицу и убедиться, что нужные столбцы доступны;
2. выделить признаки X и числовой результат y;
3. разделить данные на обучающую и тестовую части;
4. обучить LinearRegression на X_train и y_train;
5. получить predictions для объектов из X_test;
6. сравнить реальные и предсказанные значения по одним и тем же строкам;
7. посчитать абсолютную ошибку каждого прогноза и среднюю ошибку по тестовой выборке.

X обычно содержит несколько столбцов признаков, а y — один целевой столбец. После разделения индексы train относятся к данным для обучения, а индексы test — к данным для проверки.

Фрагменты кода

Сначала библиотека pandas загружает CSV-файл в DataFrame. Затем из таблицы отдельно выделяются два признака и целевая переменная.

```
import pandas as pd

df = pd.read_csv('game_scores_regression.csv')

X = df[['training_minutes', 'previous_score']]
y = df['score']
```

Двойные квадратные скобки при создании X сохраняют табличную структуру с двумя столбцами. Для y используются одинарные квадратные скобки, потому что целевая переменная представлена одним столбцом.
Далее данные делятся на обучающую и тестовую части. Параметр test_size=0.25 выделяет для проверки 25% строк, а оставшиеся 75% используются для обучения. Параметр random_state=42 фиксирует случайное разбиение, чтобы при повторном запуске получались те же обучающие и тестовые части.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
```

Метод fit обучает модель: он получает признаки X_train и соответствующие им правильные ответы y_train. Тестовые значения X_test и y_test на этом этапе в обучение не передаются. После обучения метод predict рассчитывает прогноз для каждой строки X_test. Порядок предсказаний соответствует порядку строк в тестовой выборке, поэтому их можно сопоставить с y_test.

```
predictions = model.predict(X_test)

result = X_test.copy()
result['real_score'] = y_test.values
result['predicted_score'] = predictions.round(1)
result['error'] = abs(result['real_score'] - result['predicted_score'])

result.head(10)
```

Копия X_test дополняется тремя столбцами: фактическим результатом, округлённым прогнозом и абсолютной ошибкой. Метод round(1) округляет отображаемый прогноз до одного знака после запятой, а head(10) показывает первые десять строк результата.
Среднюю абсолютную ошибку можно получить, вычислив среднее значение столбца error. При этом важно использовать одинаковое правило вычисления: если нужна наиболее точная оценка модели, ошибку следует считать по исходным predictions до округления, поскольку округление немного изменяет разницу.

Как понимать ошибку

Обычная разница может быть положительной или отрицательной: модель способна как завысить, так и занизить результат. Модуль превращает обе ситуации в неотрицательную величину и показывает размер ошибки независимо от её направления.
Если фактический результат равен 74, а прогноз модели равен 71, абсолютная ошибка равна 3 очкам. Если прогноз равен 77, абсолютная ошибка также равна 3 очкам: направление ошибки изменилось, но её размер остался тем же.
Если средняя абсолютная ошибка равна 4.2, это означает, что модель в среднем ошибается примерно на 4.2 очка. Это не процент, а значение в тех же единицах, что и целевой столбец score.
MAE рассчитывается по всем проверяемым примерам: сначала для каждого прогноза находят абсолютную ошибку, затем складывают полученные значения и делят сумму на количество примеров. Чем меньше MAE при сравнении моделей на одной и той же тестовой выборке, тем ближе их прогнозы к фактическим значениям.
Одна средняя величина не показывает ошибку каждой отдельной строки. Даже при небольшой MAE среди прогнозов могут встречаться примеры с заметно большей ошибкой, поэтому полезно рассматривать и итоговую метрику, и таблицу отдельных результатов.

Почему прогноз не обязан совпадать с фактом

Модель видит только признаки, которые мы ей передали. Два игрока могут тренироваться одинаковое время, но отличаться концентрацией, опытом, усталостью или случайными обстоятельствами прохождения уровня. Поэтому регрессионная модель обычно даёт приблизительный прогноз, а не точное будущее число.
Кроме того, LinearRegression ищет линейную зависимость. Если реальная связь между признаками и результатом устроена сложнее, модель может описать её лишь приближённо.
Ошибка прогноза поэтому помогает оценить, насколько найденная зависимость подходит для тестовых данных.
Наличие ошибки само по себе не означает, что код работает неправильно. Важно оценивать размер ошибки в контексте целевой переменной: ошибка в 4 очка может иметь разное значение для шкалы до 10 очков и для шкалы до 1000 очков.

Итог

К концу занятия обучающиеся должны объяснять, что в регрессии модель предсказывает число, а качество прогноза проверяется через ошибку между фактическим и предсказанным значением.
Также необходимо уметь выделять X и y, разделять таблицу на обучающую и тестовую части, обучать LinearRegression, получать predictions и интерпретировать абсолютную ошибку и MAE в единицах целевой переменной.
Для проверки понимания можно определить тип следующих задач: предсказать количество очков игрока, выбрать жанр игры и оценить время прохождения уровня. В первом и третьем случаях требуется число, поэтому это регрессия; во втором требуется категория, поэтому это классификация.

Материалы к занятию

lesson_3_9_regression_sklearn.pptx (231.4 КБ)

Практическое задание. Урок 3.9 «Регрессия в Scikit-learn»

Работа выполняется в Jupyter Notebook по готовому шаблону.

Файлы для работы

- `game_scores_regression.csv` - основной датасет;
- `game_scores_new_players.csv` - новые игроки для прогноза;
- `lesson_3_9_regression_sklearn.ipynb` - ноутбук с шаблоном.

Материалы к заданию

`game_scores_new_players.csv` (92 Б)

`game_scores_regression.csv` (559 Б)

`lesson_3_9_regression_sklearn.ipynb` (48.5 КБ)